

Sistema de Persistencia Políglota - Cadena de Supermercados

Trabajo Práctico Integrador - 2.ª Entrega Tema 10: Cadena de Supermercados

Materia: Ingeniería de Datos II

Institución: Universidad Argentina de la Empresa (UADE)

Profesor: Damián Ezequiel Arnaudo

Grupo: 09

Integrantes

- Farfan, Agustin (Legajo: 1178605)
 - Nesci, Valentino (Legajo: 1176278)
 - Laginestra, Gastón Nicolás (Legajo: 1176214)
 - Medina, Juan Marco (Legajo: 1173928)
 - Grupillo, Martin (Legajo: 1180138)
-

Prerrequisitos del Sistema

Para ejecutar este proyecto, es necesario contar con los siguientes motores de bases de datos instalados y en ejecución:

- **Python** 3.9 o superior.
 - **MongoDB** (Puerto por defecto: 27017).
 - **Neo4j Desktop** (Puerto por defecto Bolt: 7687).
 - **Apache Cassandra** (Puerto por defecto: 9042). Se recomienda ejecución mediante contenedor Docker.
-

Configuración del Entorno

1. Clonar el repositorio en el directorio local.
2. Crear un entorno virtual en la raíz del proyecto:

```
python -m venv venv
```

3. Activar el entorno virtual:

```
En Windows: venv\Scripts\activate  
En Linux/Mac: source venv/bin/activate
```

4. Instalar las dependencias requeridas:

```
pip install -r requirements.txt
```

5. Asegurar la existencia de un archivo .env en la raíz del proyecto con las credenciales correspondientes. Formato requerido:

```
MONGO_URI=mongodb://localhost:27017/  
MONGO_DB_NAME=tp_supermercado  
  
NEO4J_URI=bolt://localhost:7687  
NEO4J_USER=neo4j  
NEO4J_PASS=su_contraseña  
  
CASSANDRA_NODES=127.0.0.1  
CASSANDRA_KEYSPACE=tp_supermercado
```

Instrucciones de Ejecución (Poblado de Datos)

El sistema requiere un orden estricto de poblado de datos para mantener la consistencia en la arquitectura polígota.

Paso 1: Generación de datos (TP1)

Ejecutar el script generador para crear los archivos planos de prueba:

```
python generador_datos.py
```

Este comando generará los archivos .jsonl y .csv necesarios.

Paso 2: Carga en MongoDB

Desde la consola del sistema (fuera de Python), importar los archivos JSONL utilizando mongoimport:

```
mongoimport --uri "mongodb://localhost:27017" --db tp_supermercado --collection  
Productos --file mongo_productos.jsonl --drop  
mongoimport --uri "mongodb://localhost:27017" --db tp_supermercado --collection  
Sucursales --file mongo_sucursales.jsonl --drop  
mongoimport --uri "mongodb://localhost:27017" --db tp_supermercado --collection  
Proveedores --file mongo_proveedores.jsonl --drop  
mongoimport --uri "mongodb://localhost:27017" --db tp_supermercado --collection  
Stock --file mongo_stock.jsonl --drop  
mongoimport --uri "mongodb://localhost:27017" --db tp_supermercado --collection  
Tickets --file mongo_tickets.jsonl --drop  
mongoimport --uri "mongodb://localhost:27017" --db tp_supermercado --collection  
Promociones --file mongo_promociones.jsonl --drop
```

Paso 3: Carga en Neo4j

Mover los archivos .csv generados a la carpeta import de la base de datos en Neo4j Desktop. Luego, desde Neo4j Browser, ejecutar las consultas Cypher (LOAD CSV) correspondientes para establecer los nodos y relaciones.

Paso 4: Creación de Esquema en Cassandra

Ingresa a la consola de comandos de Cassandra (cqlsh) y ejecutar el archivo DDL provisto para crear el Keyspace y las tablas:

```
SOURCE '/ruta/absoluta/a1/proyecto/cassandra_schema.cql'
```

Paso 5: Poblado en Cassandra (Seeder)

Con los datos maestros ya cargados en MongoDB y el esquema de Cassandra creado, ejecutar el inyector asíncrono para poblar las tablas tabulares históricas:

```
python cassandra_seeder.py
```

Uso de la Aplicación

Una vez finalizado el poblado de los tres motores, el sistema está listo para ser evaluado:

1. Verificación de estado de motores:

```
python test_conexiones.py
```

2. Consultas operativas nativas (Sección 3 de la consigna):

Ejecuta las operaciones de alta velocidad, métricas horarias y control de stock directamente sobre Cassandra:

```
python consultas_cassandra.py
```

3. Capa Políglota y Menú Interactivo (Secciones 4 y 5 de la consigna):

Inicia la interfaz de consola que demuestra la interacción coordinada entre MongoDB, Neo4j y Cassandra:

```
python main.py
```